Reviewer Pack — Minimal p-Adic Order Correlation with Resolution Proof Width

Entry 9eac0af1428a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 01:15:34 UTC

Minimal p-Adic Order Correlation with Resolution Proof Width

Entry ID: `9eac0af1428a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 01:15:34 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Analysis **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every k-SAT instance φ with n variables, the minimal p-adic order of the coefficient associated with its clause indicator polynomial modulo a prime p is linearly correlated with its resolution proof width w(φ), such that min_p |α(p)| = Θ(w(φ)) for all primes p ≤ log(n).

Rationale (proposer's reasoning):

> p-Adic analysis provides a framework to study the arithmetic properties of polynomials. If the minimal p-adic order reflects the complexity of constructing resolution proofs, this could uncover an arithmetic-based approach to complexity theory.

Taxonomy category: `p-adic_analysis_resolution_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d22ad5ea217f2e55`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a k-SAT instance φ, if the Pearson correlation coefficient between min_p |α(p)| and w(φ) is ≥ 0.7 for all primes p ≤ log(n), then support the conjecture; otherwise, falsify it.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_k_sat_instance(n, m):
    instance = []
    for _ in range(m):
        clause = set()
        while len(clause) < 3:
            literal = random.randint(1, n)
            if random.choice([True, False]):
                literal = -literal
            clause.add(literal)
        instance.append(list(clause))
    return instance

def generate_primes(limit):
    sieve = [True] * (limit + 1)
    for start in range(2, int(math.sqrt(limit)) + 1):
        if sieve[start]:
            for multiple in range(start*start, limit + 1, start):
                sieve[multiple] = False
    return [num for num, is_prime in enumerate(sieve) if is_prime and num > 1]

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def p_adic_order(coeff, prime):
    order = 0
    abs_coeff = abs(coeff)
    while abs_coeff % prime == 0:
        abs_coeff //= prime
        order += 1
    return order

def resolution_width(instance):
    # Simplified version of resolution width calculation
    # This is a placeholder and should be replaced with actual logic
    return len(instance)
```

```
def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    m = random.randint(n * 2, n * 3)
    instance = generate_k_sat_instance(n, m)

    primes = generate_primes(int(math.log(n)) + 1)
    if not primes:
        return {
            "metric_name": "p-adic order correlation",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "no_primes_found"
        }

    p_adic_orders = []
    for prime in primes:
        coeff = random.randint(-100, 100)
        order = p_adic_order(coeff, prime)
        p_adic_orders.append(order)

    width = resolution_width(instance)

    if not p_adic_orders or width == 0:
        return {
            "metric_name": "p-adic order correlation",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "invalid_resolution_width"
        }

    mean_order = sum(p_adic_orders) / len(p_adic_orders)
    correlation_coefficient = (sum((x - mean_order) * (y - width) for x, y in zip(p_adic_orders, p
                               (len(p_adic_orders) * math.sqrt(sum((x - mean_order) ** 2 for x in
                                math.sqrt(sum((y - width) ** 2 for y in p_adic_orders))))

    return {
        "metric_name": "p-adic order correlation",
        "metric_value": correlation_coefficient,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(correlation_coefficient) >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
```

```
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = "unknown"
        print(f"RESULT: FALSIFIED counterexample='{counterexample}' first_failing_seed={first_fail
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'p-adic order correlation', 'metric_value': 0.010750202576038453, 'instances_

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2fd17f5f.py", line 112, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2fd17f5f.py", line 93, in run_trial
    correlation_coefficient = (sum((x - mean_order) * (y - width) for x, y in zip(p_adic_orders, p_adic_
                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means it did not complete successfully. The Pearson correlation coefficient was not calculated as the test failed to execute. | next: Re-run the test ensuring that it completes without crashing and produces a valid result.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13092
2	preregistration	ollama_remote	glm4:latest	0	0	26917

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	8358
4	novelty	ollama_remote	glm4:latest	0	0	8568
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17091
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10972
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30601
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21957
9	judge	ollama_remote	glm4:latest	0	0	11787

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 149342 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9eac0af1428a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9eac0af1428a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9eac0af1428a.tar.gz` (if generated)